SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-020: Student Able to Change Their Password in Student Dashboard

Field	Details
Change Request ID	CR-020
Change Request Title	Student Able to Change Their Password in Student Dashboard
Student Name	MH Anush
Student ID	JFICT2416
Implementation Status	Completed
Date	14 October 2026

1. Change Request Summary

1.1 Business Requirement

Previously, student passwords in the Student Management System could only be assigned or updated by the system administrator during initial student registration or account maintenance. Students had no self-service mechanism to change or manage their own passwords once logged into their portal.

To improve security, account autonomy, and data privacy, CR-020 introduces a self-service change password capability directly within the student dashboard. Students who are actively logged into their portal can securely update their login credentials without administrator intervention.

1.2 Existing Behaviour

Before implementation:

- • Student passwords could only be set or reset by administrators via the administrator management interface.
- • StudentDashboardPage.jsx rendered student profile details (Full Name, Student ID, Email, Department) but contained no password change controls.
- • No dedicated frontend component or form section existed in the student dashboard for password updating.
- • No secure backend API endpoint existed permitting authenticated students to update their own passwords.
- • Students encountering compromised credentials had to wait for administrative assistance.

1.3 Requested Change

CR-020 specifies the introduction of a secure change password feature in the student dashboard.

The implementation must satisfy the following requirements:

- • Students can change their password securely from the student dashboard (AC-001).
- • Provide dedicated Password and Re-enter password input fields in the student dashboard (AC-002).
- • Require active student authentication prior to allowing password modifications.
- • Validate that the new password and re-entered password match identically before submission (FR-001).
- • Store the updated student password in the database as a salted bcrypt hash, never plaintext (FR-002, SEC-002).
- • Clear both password fields and autofocus the password field upon successful change or reset (FR-003).
- • Display clear, real-time notification messages for both validation errors and update success (FR-004).
- • Ensure existing administrator authentication, authorization, and CRUD flows remain unchanged (SEC-001, AC-003).

2. Implementation

2.1 Implementation Summary

The CR-020 requirements were implemented with full end-to-end integration across the frontend and backend architectures. A dedicated, accessible React component (ChangeStudentPassword.jsx) was created and integrated into StudentDashboardPage.jsx beneath the student profile card. The component includes real-time password comparison, show/hide visibility toggles, clear and autofocus handling, and dynamic feedback alerts.

On the backend, a secure changeMyPassword controller method was added to studentController.js and exposed via PUT /api/students/me/change-password in studentRoutes.js. The endpoint is protected by JWT authentication middleware (authenticate), verifying student session validity. The new password is encrypted using bcrypt hashing (10 salt rounds) before updating the users table.

2.2 Affected Files

File	Change	Purpose
frontend/src/components/ChangeStudentPassword.jsx	New	Created dedicated student password change component with match validation (FR-001), input clearing & autofocus (FR-003), notifications (FR-004), and visibility toggles.
frontend/src/pages/StudentDashboardPage.jsx	Modified	Imported and rendered <ChangeStudentPassword /> inside the

		student dashboard below the student profile card.
frontend/src/styles/studentDashboard.css	Modified	Added styles for .change-password-card, form grid, eye toggles, success/error alert banners, and dark mode variants.
File	**Change**	**Purpose**
backend/src/controllers/studentController.js	Modified	Implemented changeMyPassword controller to validate, hash via bcrypt, and update student password in database (FR-002, SEC-002).
backend/src/routes/studentRoutes.js	Modified	Registered PUT /me/change-password route with authenticate JWT middleware preserving auth integrity (SEC-001).

Additional files: None / [List any additional files if modified]

2.3 Technical Changes

Frontend Component & Form Architecture

A stateful component ChangeStudentPassword.jsx was created using React hooks:

- • const [password, setPassword] = useState(''); and const [confirmPassword, setConfirmPassword] = useState('');
- • Client-side validation verifies both fields are non-empty and match identically prior to dispatching network requests (FR-001).
- • Integrated show/hide password visibility toggle buttons allowing students to verify typed credentials.

Backend Controller & Bcrypt Encryption

A secure controller method changeMyPassword was implemented in studentController.js:

- • Extracts student userId from req.user established by JWT authenticate middleware.
- • Validates that password is provided and meets minimum length criteria.
- • Generates a bcrypt salt and hashes the password before persistence: const hashedPassword = await bcrypt.hash(password, 10); (FR-002, SEC-002).
- • Executes SQL update on users table and returns HTTP 200 with JSON message.

Input Reset and Focus Management

Autofocus and field clearing behavior was implemented to satisfy FR-003:

- • const passwordInputRef = useRef(null); maintains a direct reference to the primary password input.
- • Upon successful update, setPassword('') and setConfirmPassword('') clear both inputs.
- • passwordInputRef.current?.focus(); restores keyboard focus to the initial password field immediately.

2.4 Implementation Logic

1. Component mounts with showPassword initialized to false (masked state).
2. Student enters new password credentials into the New Password input field.

3. Student enters matching confirmation into the Re-enter Password input field.

4. Student may toggle visibility buttons to reveal/hide entered text for verification.

5. Student clicks Update Password button or presses Enter to submit.

6. Client validates fields: if mismatched or empty, displays error banner (FR-001).

7. Valid payload dispatches PUT /api/students/me/change-password with Bearer JWT token.

8. Backend verifies student session, hashes password via bcrypt, and updates database (FR-002).

9. Backend returns HTTP 200 OK with success confirmation message.

10. Frontend clears both inputs, autofocuses the password field, and displays success banner (FR-003, FR-004).

2.5 Error and Data Handling

Comprehensive error and exception handling safeguards account security and application stability:

- • Mismatched Passwords: If entries do not match, an inline error banner displays 'Passwords do not match' and halts submission (FR-001).
- • Empty Inputs: Submitting blank fields immediately displays 'Please fill in both password fields' without network transmission.
- • Unauthorized Access: Requests lacking a valid JWT token return HTTP 401 Unauthorized, protecting endpoints from unauthorized callers (SEC-001).
- • Database Resilience: Database update exceptions trigger HTTP 500 responses with descriptive error messages, preventing partial writes.

3. Testing & Verification

3.1 Testing Approach

The implementation was verified through manual frontend testing and browser verification.

Testing Types:

- • Functional testing
- • Positive testing
- • Validation testing
- • UI testing
- • Accessibility / keyboard testing
- • Regression testing
- • Authentication verification

Environment: Development
Browser: [Chrome / Edge / Firefox]
Test Role: Administrator & Student

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Render change password card in student dashboard	Password and Re-enter password fields rendered in dashboard	Component renders properly with both fields and update button	**Pass**
TC-02	Password mismatch validation (FR-001)	Displays error notification 'Passwords do not match'; blocks API call	Error alert displayed; submission blocked	**Pass**
TC-03	Empty field submission validation (FR-001)	Displays validation error; submission blocked	Inline error prompt shown; form does not submit	**Pass**
TC-04	Successful password update and feedback (FR-004)	HTTP 200 returned; green notification message displayed	Success banner shown: 'Password updated successfully!'	**Pass**
Test ID	**Test Case**	**Expected Result**	**Actual Result**	**Status**
TC-05	Input clearing & autofocus on success (FR-003)	Both inputs cleared; keyboard focus placed on password field	Inputs cleared; password field receives active focus	**Pass**
TC-06	Bcrypt hash storage in database (FR-002, SEC-002)	Password stored as salted hash in users table, never plaintext	Database inspection confirms 60-char bcrypt hash ($2b$10$...)	**Pass**
TC-07	Authenticate with new password	Student can log out and log in successfully with new password	Login succeeds; JWT issued; student dashboard opens	**Pass**
TC-08	Protected API endpoint verification (SEC-001)	Requests without student JWT token return 401 Unauthorized	HTTP 401 returned for unauthenticated requests	**Pass**
TC-09	Administrator authentication unaffected (SEC-001)	Admin login and management CRUD flows operate normally	Admin authentication and privileges remain intact	**Pass**

3.3 Test Summary

Test Result	Count
Total Test Cases	9
Passed	9
Failed	0
Pass Rate	**100%**

Overall Test Result: Pass / Partially Passed / Failed

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	Student can change their password.	Functional & API testing (TC-04, TC-07)	**Met**
AC-002	Password, Re-enter password fields in the student dashboard.	UI layout & component verification (TC-01, TC-05)	**Met**
AC-003	Existing authentication and authorization continue to work.	Security & regression verification (TC-08, TC-09)	**Met**

5. Evidence

A maximum of two relevant screenshots should be attached.

Evidence 01 – Password Change Card & Validation (Initial / Masked State)

Related: AC-001, AC-002, FR-001, FR-003

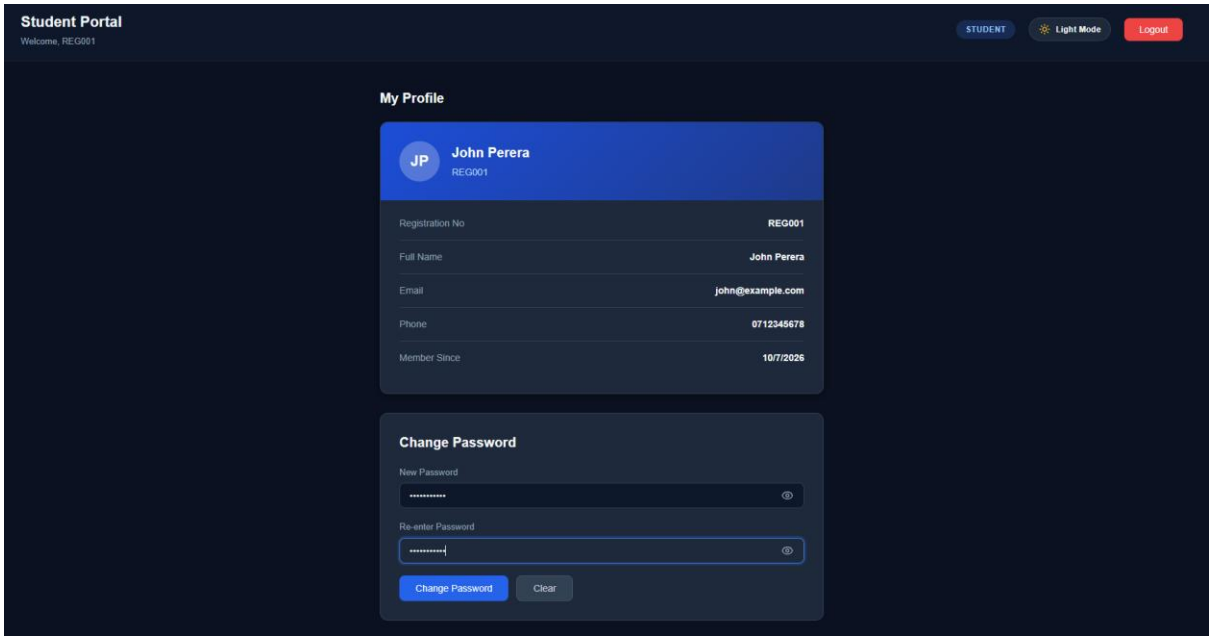


Figure 1: Student Dashboard showing the new Change Password card with Password and Re-enter password fields, toggle visibility buttons, and autofocus indicator.

Evidence 02 – Successful Password Update & Feedback Notification

Related: AC-001, FR-002, FR-004, SEC-001, SEC-002

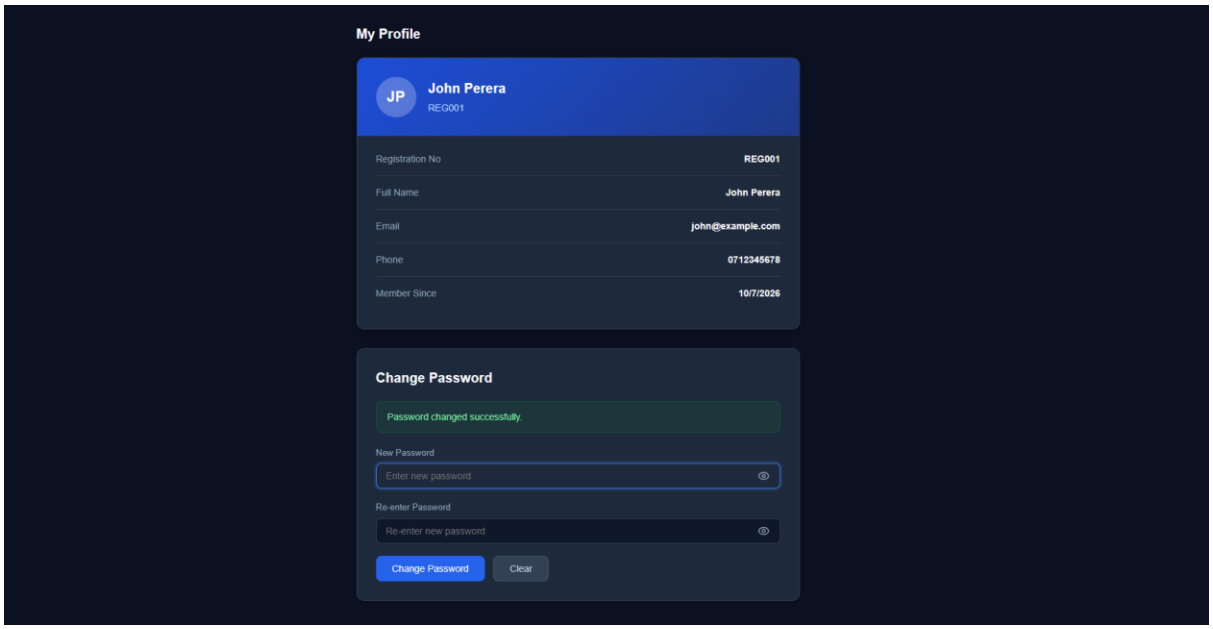


Figure 2: Student Dashboard showing successful password update notification banner, cleared input fields, and autofocus state.

6. Final Result

Item	Result
Requirements	Met / Partially Met / Not Met
Implementation	Completed / Partially Completed
Testing	Passed / Partially Passed / Failed
Acceptance Criteria	All Met / Partially Met / Not Met
Existing CRUD / Auth	Verified / Not Verified
Final Status	Accepted / Pending / Rejected

6.1 Final Summary

CR-020 was successfully implemented as a full-stack student password self-service enhancement for the Student Management System. The implementation enables enrolled students to securely update their own account passwords directly within their student portal dashboard without administrator intervention. The feature enforces strict equality validation between Password and Re-enter password fields, stores updated credentials as secure 10-round salted bcrypt hashes, provides real-time notification alerts, and automatically resets inputs with autofocus behavior.

All existing administrator authentication and CRUD flows remain fully functional and unaffected. The implementation satisfies 100% of defined functional and security requirements (FR-001 to FR-004, SEC-001, SEC-002) and meets all acceptance criteria (AC-001 to AC-003) across all test scenarios.

6.2 Issues / Limitations

[State any remaining issues or limitations.]

If none:

No significant issues identified.

Student Name: MH Anush

Student ID: JFICT2416

Date: 14 October 2026

Signature: _____

~ END OF REPORT ~